

PROJECT-BASED LEARNING

PHASE-I EVALUATION

IntelliPlag

Team ID: DSCPP-III-2026-T069

Department of Computer Science & Engineering

Graphic Era (Deemed to be University), Dehradun

Academic Session 2026-27

Team & Mentor Details

Team Information

Team Name: IntelliPlag
Team ID: DSCPP-III-2026-T069
Semester: 3rd
Domain: Software Development

Team Members

1. Shivya Sinha – 2510380257
2. Srishti Kuletha – 2510012545
3. Antriksh Singh Jagwan – 2512510006

Mentor

Srishti Chamoli

Interactions completed: _____

Problem Statement & Motivation

Problem Statement

With the increasing use of digital documents and online academic content, identifying copied or highly similar text manually has become difficult and time-consuming. Existing plagiarism checkers may provide similarity scores, but the underlying comparison process is often not visible to students.

IntelliPlag aims to develop a plagiarism detection system that processes documents, generates text fingerprints using data structures and algorithms, identifies matching phrases or sections, and produces an understandable plagiarism report.

Motivation

- ▶ Manual comparison of large documents is inefficient.
- ▶ Similar or copied sections need to be identified automatically.
- ▶ Students need an understandable similarity report.
- ▶ DSA concepts can be applied directly to text comparison.

Target Users / Stakeholders

- ▶ Students
- ▶ Faculty members
- ▶ Academic evaluators
- ▶ Educational institutions

Objectives

Project Objectives

1. Develop a system for detecting textual plagiarism between documents.
2. Preprocess and tokenize input documents for efficient comparison.
3. Apply DSA concepts such as Tries, Hash Tables and Vectors for text representation and fingerprinting.
4. Generate N-Grams and perform phrase/string matching to identify similar sections.
5. Calculate an overall similarity percentage and rank detected matches according to their relevance.
6. Generate a clear and structured plagiarism report.

Key Objective: Automatically identify matching content between documents and provide a meaningful plagiarism similarity score.

Proposed Solution

Proposed Approach

1. Input documents are provided to the system.
2. Text is preprocessed by converting it to lowercase and removing unnecessary punctuation.
3. The text is tokenized into individual words/tokens.
4. DSA modules store and process the tokens using Tries, Hash Tables and Vectors.
5. N-Grams are generated from the processed text.
6. Phrase/string matching identifies common sections.
7. Similarity is calculated from the detected matches.
8. A plagiarism report is generated.

Key Features

- ▶ Document text preprocessing
- ▶ Tokenization
- ▶ Trie-based word processing
- ▶ Hash-based text fingerprinting
- ▶ N-Gram generation
- ▶ Phrase and string matching
- ▶ Similarity percentage calculation
- ▶ Matching-section identification
- ▶ Priority/ranking of detected matches
- ▶ Plagiarism report generation

The system focuses on algorithmic plagiarism detection using data structures and core string-processing techniques.

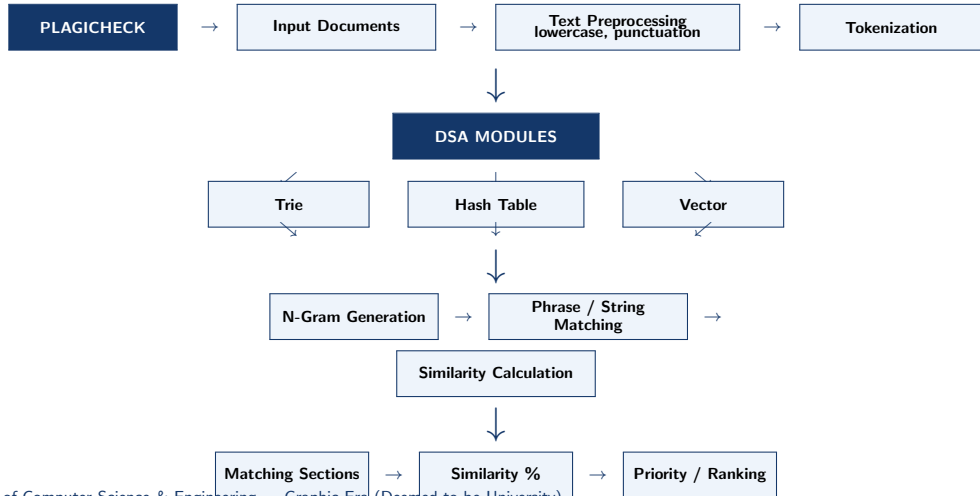
Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Arrays, Vectors, Hash Tables, Tries, searching, string processing and N-Grams	Text Processing & Similarity Engine
OOPs with C++	Classes, objects, encapsulation, inheritance, polymorphism and modular design	Document & Detection Modules
Machine Learning	Feature extraction and optional classification/ similarity prediction	Similarity Analysis
DBMS	Document storage, analysis results and retrieval	Database Layer

Core Requirement:

Meaningful application of Data Structures and OOP concepts in the plagiarism detection engine.

System Architecture / Workflow



Technology Stack & Methodology

Technology Stack

- ▶ Core DSA: **C**
- ▶ OOP Implementation: **C++**
- ▶ ML / Data Processing: **Python**
- ▶ Frontend: **HTML, CSS, JavaScript**
- ▶ Database: **MySQL**
- ▶ Development Environment: **VS Code**
- ▶ Version Control: **Git / GitHub**

Development Methodology

1. Requirement Analysis
2. Document Processing Design
3. DSA Module Implementation
4. Similarity Algorithm Development
5. OOP Integration
6. ML / Database Integration
7. UI Development
8. Testing and Evaluation

The core detection process is based on DSA and string-processing algorithms, while ML, database and UI components

support the system.

Initial Progress & Team Contribution

Progress Achieved

- ▶ Project problem and scope identified
- ▶ Plagiarism detection pipeline prepared
- ▶ Text preprocessing approach defined
- ▶ Tokenization workflow identified
- ▶ DSA modules selected: Trie, Hash Table and Vector
- ▶ N-Gram based comparison approach planned
- ▶ Similarity calculation and reporting workflow defined

Team Contribution

Member	Role	Contribution
Shivya	Core / DSA	DSA implementation and plagiarism detection algorithms
Srishti	OOP / UI	OOP integration, interface design and system integration
Antriksh	ML / DB	ML integration, database handling and testing

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal & System Design
2. Phase-II: DSA and OOP Implementation
3. Phase-III: Similarity Engine and Integration
4. Phase-IV: Testing and Final Implementation

Expected Outcomes

- ▶ Functional plagiarism detection system
- ▶ Automatic identification of matching sections
- ▶ Similarity percentage for compared documents
- ▶ Priority/ranking of detected matches
- ▶ Clear plagiarism report
- ▶ Practical demonstration of DSA and OOP

References

1. T. H. Cormen et al., *Introduction to Algorithms*, MIT Press.
2. B. Stroustrup, *The C++ Programming Language*, Addison-Wesley.
3. M. T. Goodrich et al., *Data Structures and Algorithms*, Wiley.
4. Research literature on plagiarism detection, N-Gram analysis and text similarity.
5. Documentation for selected ML algorithms and database technologies.

Thank You
Questions & Suggestions